

UNIDAD ACADÉMICA MULTIDISCIPLINARIA MANTE - CENTRO

NOMBRE DE LOS ALUMNOS:

JESUS ALBERTO LOREDO CERVANTES
LEON BANDA BRAYAN ALEJANDRO
QUEZADA RUIZ EMILY ALEJANDRA
CASTILLO SERRATOS JUAN ANTONIO
CORTEZ MARTINEZ JORGE ALBERTO
NUÑEZ MENCHACA ALEJANDRO EMILIANO
GALICIA MARQUEZ GABRIELA RUBI

NOMBRE DE LA MATERIA:

FUNDAMENTOS DE
PROGRAMACION

NOMBRE DEL DOCENTE : MARIA SABINA MUÑIZ MONTOYA

GRADO Y GRUPO: 1.-E

LUGAR Y FECHA :

CD. MANTE

TAMAULIPAS A 8 DE

SEPTIEMBRE DEL

2026

INDICE

UNIDAD I INTRODUCCIÓN A LA ALGORITMIA

Introducción (Pag. 4)

1. Introducción a la Algoritmia (Pag. 5-25)

1.1. La Computabilidad y conceptos de algoritmo (Pag. 5)

1.2. Elementos de los algoritmos y tipos de datos (Pag. 6-7)

1.3. Representación de los algoritmos en diagrama de flujo (Pag. 8-9)

1.4. Estructuras básicas: secuencial, condicional e iteración (Pag. 10-11)

1.5. Resolución de problemas básicos (Pag. 12)

Primeros 5 ejemplos (Pag. 13-22)

Segundos 2 ejemplos (Pag. 23-25)

Conclusion (Pag. 26)

Fuentes (Pag. 27)

INTRODUCCIÓN

La algoritmia es una parte fundamental de la informática que se encarga de estudiar y desarrollar métodos ordenados para resolver problemas de manera eficiente. Un algoritmo consiste en una serie de pasos lógicos y finitos que permiten llegar a una solución determinada a partir de ciertos datos o instrucciones.

En la vida cotidiana utilizamos algoritmos constantemente, aunque no siempre somos conscientes de ello, por ejemplo, al seguir una receta de cocina o las instrucciones para realizar una actividad.

Comprender los fundamentos de la algoritmia permite desarrollar habilidades como el razonamiento lógico, la organización y la capacidad para resolver problemas.

Además, es un conocimiento esencial para aprender programación, ya que antes de escribir un programa es necesario establecer claramente el procedimiento que se seguirá para obtener el resultado esperado.

Por ello, la introducción a la algoritmia representa el primer paso para comprender cómo funcionan los programas y cómo la tecnología puede utilizarse para solucionar diferentes tipos de problemas.

Introducción a la Algoritmia

1.1.-La Computabilidad y conceptos de algoritmo

Un algoritmo informático es un conjunto de instrucciones definidas, ordenadas y acotadas para resolver un problema, realizar un cálculo o desarrollar una tarea. Es decir, un algoritmo es un procedimiento paso a paso para conseguir un fin.

Las tres partes de un algoritmo son:

1. **Input (entrada).** Información que damos al algoritmo con la que va a trabajar para ofrecer la solución esperada.
2. **Proceso.** Conjunto de pasos para que, a partir de los datos de entrada, llegue a la solución de la situación.
3. **Output (salida).** Resultados, a partir de la transformación de los valores de entrada durante el proceso.

De este modo, un algoritmo informático parte de un estado inicial y de unos valores de entrada, sigue una serie de pasos sucesivos y llega a un estado final en el que ha obtenido una solución.

Características de los algoritmos

Asimismo, los algoritmos presentan una serie de características comunes. Son:

- Precisos. Objetivos, sin ambigüedad.
- Ordenados. Presentan una secuencia clara y precisa para poder llegar a la solución.
- Finitos. Contienen un número determinado de pasos.
- Concretos. Ofrecen una solución determinada para la situación o problema planteados.

1.2.-Elementos de los algoritmos y tipos de datos

ELEMENTOS

Entrada: Se trata del conjunto de datos que el algoritmo necesita como insumo para procesar.

Proceso: Son los pasos necesarios aplicados por el algoritmo a la entrada recibida para poder llegar a una salida o resolución del problema.

Salida: Es el resultado producido por el algoritmo a partir del procesamiento de la entrada una vez terminada la ejecución del proceso.

TIPOS DE DATOS

1- Datos numéricos

Los datos numéricos son probablemente los tipos de datos más utilizados en programación. Incluyen varios subtipos que permiten la representación de números en diferentes formas y precisiones.

2- Datos booleanos

Los datos booleanos son otro tipo de dato esencial en programación. Así pues, representan valores de verdad y solo pueden tener uno de dos posibles valores: true (verdadero) o false (falso).

3.1- Datos de texto

Los datos de texto también conocidos como cadenas de caracteres o strings, son secuencias de caracteres que representan texto. Este tipo de dato es ampliamente utilizado para manipular y almacenar información textual.

3.2- Datos de colección

Los datos de colección son tipos de datos que permiten almacenar múltiples valores en una sola variable. Por lo tanto, son esenciales para manejar grandes volúmenes de datos de manera organizada y eficiente.

Metadatos

En esencia, los metadatos son datos que describen otros datos. Es decir, proporcionan información adicional sobre los datos principales, como su origen, formato, contexto, y otros atributos importantes. Así pues, este tipo de información es esencial para la organización, gestión y comprensión de los datos en diversos sistemas y aplicaciones.



Ilustración 1. Datos en un algoritmo.

1.3.-Representación de los algoritmos en diagrama de flujo

DIAGRAMA DE FLUJO

Los diagramas de flujo son herramientas valiosas en la conceptualización, diseño y análisis de algoritmos. Facilitan la visualización de la estructura lógica de un programa, permitiendo identificar rápidamente las relaciones entre sus componentes y simplificando el proceso de depuración y optimización. Son especialmente útiles en la fase de diseño de nuevos programas, mejoran la comunicación entre desarrolladores y usuarios finales y proporcionan una documentación visual que puede ser de gran ayuda durante la codificación y las pruebas del software. No obstante, los diagramas de flujo tienen limitaciones, particularmente en el caso de algoritmos extensos o complejos, donde pueden volverse difíciles de seguir y entender. La creación de un diagrama de flujo para un algoritmo complejo puede ser un proceso tedioso y propenso a errores, y la falta de estándares universales para su elaboración puede resultar en interpretaciones ambiguas o en la omisión de detalles importantes para la comprensión del algoritmo.

Al crear un diagrama de flujo, es esencial seguir reglas de diseño claras, como la orientación de arriba hacia abajo y de izquierda a derecha, para garantizar la legibilidad y la correcta interpretación del algoritmo.

Las representaciones visuales que se utilizan en los diagramas de flujo se componen de símbolos estandarizados para mostrar los pasos de un proceso de manera secuencial y lógica. Estos símbolos incluyen óvalos para el inicio y fin, rectángulos para las operaciones, rombos para las decisiones, entre otros, y se conectan con flechas que indican el flujo de operaciones.

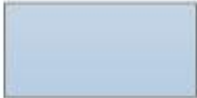
Símbolo	Nombre	Función
	Inicio / Final	Representa el inicio y el final de un proceso
	Linea de Flujo	Indica el orden de la ejecución de las operaciones. La flecha indica la siguiente instrucción.
	Entrada / Salida	Representa la lectura de datos en la entrada y la impresión de datos en la salida
	Proceso	Representa cualquier tipo de operación
	Decisión	Nos permite analizar una situación, con base en los valores verdadero y falso

Ilustración 2. Simbología del diagrama de flujo.

1.4.-Estructuras básicas: secuencial, condicional e iteración

ESTRUCTURAS BÁSICAS

Los algoritmos pueden tener diferentes opciones de resultado, dependiendo de los datos de entrada que se proporcionen o del resultado de un determinado proceso. Para lograr que un algoritmo llegue a la solución, se pueden usar distintas estructuras. Las estructuras básicas de los algoritmos son: *Secuencial*, *Condicional* y *iteración*.

Estructura secuencial.

A diario realizamos actividades donde están presente definiciones que desconocemos, o manejamos equipos donde olvidamos que, sin esas prácticas conceptuales, sería casi imposible el desarrollo de esas funciones de las que nos beneficiamos a través de un sistema basado en la informática.

¿Qué es la estructura secuencial?

Es el conjunto de movimientos en la que una acción sigue a otra (tal como su nombre lo indica), mediante una secuencia que sigue a una operación programada. Al igual que otras estructuras realizadas en un sistema, tienen una entrada y una salida.

Generalmente, el proceso que desempeña una estructura secuencial consta de un comienzo, seguido por la ejecución número 1, luego la ejecución número dos, posteriormente la ejecución número tres, hasta llegar a la finalización del proceso. Este esquema de trabajo contempla un diagrama de flujo que a su vez está compuesto por un grupo de pseudocódigos programables.

Un ejemplo de un diagrama de flujo en una estructura secuencial es el proceso que todos hacemos al hacer una llamada telefónica. El paso número uno es levantar la bocina, luego esperamos tono, seguidamente marcamos el número, esperamos que contenten; hablamos con la persona, para finalmente colgar la bocina que vendría siendo el final de todo el proceso.

Estructuras condicionales. Estas herramientas son fundamentales para cualquier programador, ya que te permiten tomar decisiones dentro de tu código. Son instrucciones que le indican a tu software qué camino seguir según ciertas condiciones. Imagina que estás en un cruce de caminos y debes decidir si ir a la izquierda o a la derecha dependiendo de si tienes un mapa. De la misma manera, una estructura condicional permite que tu programa ejecute un bloque de código si se cumple una condición específica y otro bloque si no se cumple.

(Toma de decisiones)

Permiten ejecutar un bloque de código u otro según se cumpla o no una condición.

- **if (si):** Ejecuta un bloque de código solo si una condición resulta verdadera (true).
- **else (si no):** Se usa junto a if para ejecutar un bloque alternativo cuando la condición del if resulta falsa (false).
- **switch (según / interruptor):** Compara el valor de una variable contra múltiples opciones (casos) para elegir qué camino de código ejecutar. Es más ordenado que usar muchos if anidados.

Iterativa: Una estructura iterativa o de repetición es aquella que permite que un proceso o conjunto de acciones se realice de manera cíclica (loop o bucle). Este proceso repetitivo se realiza mientras una condición lógica sea verdadera. Existen cuatro formas comunes de este tipo de estructuras: "Hacer mientras" (Do-While), "Mientras que", "Repite hasta que" y "Desde, hasta" (For-next). En este apartado sólo trabajaremos con la primera forma, pues es la que usa DescartesJS. En la figura de la izquierda se presenta la forma Do-While en un diagrama de flujo.

Importancia: La estructura iterativa (o bucle) es fundamental en programación porque permite repetir un conjunto de instrucciones múltiples veces, lo que proporciona eficiencia, claridad y control sobre la ejecución de un programa. Un aspecto a destacar es la automatización de tareas repetitivas, pues en lugar de escribir el mismo código varias veces, una estructura iterativa ejecuta una acción repetidamente.

1.5 Resolución de problemas básicos.

Para solucionar un problema por medio de un algoritmo, se debe primero entender que es un algoritmo:

Algoritmos

Un **algoritmo** es una serie de pasos que resuelve un problema algorítmico. Esta serie de pasos debe resolver todas las posibles instancias, sin importar cual sea. Mediante un cierto proceso, debemos explicar poco a poco cómo podemos tomar la entrada, procesarla y dar una salida que resuelva el problema con las condiciones establecidas.

Ahora deben seguirse esta serie de pasos para poder estructurar y diseñar un algoritmo de manera más clara y específica:

Analizar el problema: consiste en comprender el enunciado del problema, identificar los datos de entrada, los datos de salida y las restricciones o condiciones que se deben cumplir.

Diseñar el algoritmo: consiste en elaborar una solución lógica al problema, utilizando las variables, las operaciones y las estructuras de control necesarias. Se puede utilizar un lenguaje natural, un pseudocódigo o un diagrama de flujo para representar el algoritmo.

Verificar el algoritmo: consiste en comprobar que el algoritmo sea correcto y cumpla con los objetivos del problema. Se puede utilizar ejemplos, casos de prueba o tablas de traza para verificar el algoritmo.

Implementar el algoritmo: consiste en traducir el algoritmo a un lenguaje de programación específico, que pueda ser ejecutado por una computadora. Se debe seguir la sintaxis y la semántica del lenguaje elegido y depurar posibles errores.

Evaluar el algoritmo: consiste en medir el rendimiento del algoritmo, tanto en términos de eficiencia (tiempo y espacio) como de efectividad (calidad y precisión). Se puede utilizar indicadores, métricas o criterios para evaluar el algoritmo.

EJERCICIOS EN PSeInt Y C

PSeInt:

Algoritmo AREA_RECTANGULO

```
definir area, base, altura Como Real
escribir "ingrese la base del rectangulo"
leer base
escribir "ingrese altura del rectangulo"
leer altura
area = base * altura
escribir "el area del rectangulo es:" area
```

C:

```
#include <stdio.h>
```

```
int main() {
    float base, altura, area;
    printf("Ingrese la base: ");
    scanf("%f", &base);
    printf("Ingrese la altura: ");
    scanf("%f", &altura);
    area = base * altura/2;
    printf("La area es: %.2f", area);
    return 0;
}
```

FinAlgoritmo

Se definieron primero las variables de las proporciones del rectángulo como valores reales para después poder solicitarlas, posteriormente se escribió la formula correspondiente para el cálculo del área la cual es “base * altura” y finalmente se escribió el resultado para poder cerrar el algoritmo

```
PrecioDeUnProductoConIVAPSE.psc AreaRectanguloPSE.psc X
1 Algoritmo CalcularAreaRectangulo
2   definir base, altura, area como real
3
4   escribir "Ingresa la base : "
5   leer base
6
7   Escribir "Ingresa la altura : "
8   leer altura
9
10  area = base * altura
11
12  Escribir "La area es igual a : " area
13
14
15 FinAlgoritmo
```

```
PSeInt - Ejecutando proceso CALCULARAREAR...
*** Ejecución Iniciada. ***
Ingresa la base :
> 8
Ingresa la altura :
> 2
La area es igual a : 16
*** Ejecución Finalizada. ***
☐ No cerrar esta ventana ☐ Siempre visible 
```

Ilustración 3. Algoritmo AREA_RECTANGULO en PSeInt.

```
AreaDeUnRectanguloC.c X
1 #include <stdio.h>
2
3 int main() {
4     float base, altura, area;
5
6     printf(format: "Ingresa la base: ");
7     scanf(format: "%f", &base);
8     printf(format: "Ingresa la altura: ");
```

```
"D:\mench\Descargas\Ejercicios\EN C\AreaDeUnRectanguloC.exe"
Ingresa la base:5
Ingresa la altura:5
La area es: 12.50
```

Ilustración 4. Algoritmo AREA_RECTANGULO en C.

PSeint:

Algoritmo HORAS_SALARIO

definir salario, horas, dinero Como Real

Escribir "ingrese horas trabajadas:"

leer horas

Escribir "ingrese cuanto le pagan por hora:"

leer dinero

salario = horas * dinero

Escribir "las horas trabajdas:" horas

Escribir "y lo que le pagan por hora:" dinero

Escribir "equivalen a este salario:" salario

C:

```
#include <stdio.h>
```

```
int main() {
```

```
    float dinero, horas, salario;
```

```
    printf("Ingrese cuanto gana por hora : ");
```

```
    scanf("%f", &dinero);
```

```
    printf("Ingrese las horas trabajadas cada dia : ");
```

```
    scanf("%f", &horas);
```

```
    salario = dinero * horas * 5; //variacion de dias
```

```
    printf("El salario por semana es de : %.2f", salario);
```

```
    return 0;
```

```
}
```

FinAlgoritmo

Se definieron las variables de la proporcionalidad de las horas y el dinero ganado por hora como valores reales, después se solicitan los valores exactos de esas variables para poder aplicar la formula necesaria “horas * dinero” y así escribir el salario total por esas horas (en la foto se muestra como ganancia a la semana), finalmente se cerró el algoritmo

```

<sin_titulo> - Calcular_Salario_Semanal.pse X
1 Algoritmo Calcular_Salario_Semanal
2   definir dinero, Salario , horas como real
3   dinero ← 0
4   horas ← 0
5   Escribir "Ingrese cuantas horas ah trabajado en un día : "
6   LEER horas
7   Escribir "ingrese el dinero que le pagan por hora"
8   leer dinero
9   Salario ← dinero * horas * 5 //el valor de los días de la semana puede variar para este ejemplo utilizamos 5 días
10
11   Escribir "LAS HORAS:" horas
12   ESCRIBIR "DINERO POR HORA : " dinero
13   ESCRIBIR "Son equivalentes a este salario por semana:" salario
14
15
16
17
18
19 Finalgoritmo
20

```

PSeInt - Ejecutando proceso CALCULAR_SALARIO_SEMANAL

*** Ejecución Iniciada. ***

Ingrese cuantas horas ah trabajado en un día :

> 5

ingrese el dinero que le pagan por hora

> 50

LAS HORAS:5

DINERO POR HORA : 50

Son equivalentes a este salario por semana:1250

*** Ejecución Finalizada. ***

Ilustración 5. Algoritmo HORAS_SALARIO en PSeInt.

```

#include <stdio.h>

int main() {
    float dinero, horas, salario;

    printf(format:"Ingrese cuanto gana por hora : ");
    scanf(format:"%f", &dinero);

    printf(format:"Ingrese las horas trabajadas cada día : ");
    scanf(format:"%f", &horas);

    salario = dinero * horas * 5; //variacion de dias

    printf(format:"El salario por semana es de : %.2f", salario);

    return 0;
}

```

Run DineroPorHoraC.c

"D:\mench\Descargas\Ejercicios\EN C\DineroPorHoraC.exe"

Ingrese cuanto gana por hora :50

Ingrese las horas trabajadas cada día :5

El salario por semana es de : 1250.00

Process finished with exit code 0

Ilustración 6. Algoritmo HORAS_SALARIO en C.

PSeint:

Algoritmo METROS_A_CENTIMETROS

Definir centimetros, metros Como Real

escribir "ingrese la cantidad de metros:"

leer metros

centimetros = metros * 100

escribir "los metros ingresados equivalen a estos centimetros:" centimetros;
escribir "cm"

C:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    float metros, centimetros;
```

```
    printf("Porfavor ingresa los metros a calcular: ");
```

```
    scanf("%f",&metros);
```

```
    centimetros = metros * 100;
```

```
    printf("Los metros son : %f ", metros);
```

```
    printf("En centimetros son : %f ", centimetros);
```

```
}
```

FinAlgoritmo

Primero se definieron las variables metros y centímetros como reales, después se solicitó la cantidad de metros para poder calcular los centímetros con la formula “metros * 100” se escribió el resultado y se cerró el algoritmo

```
PrecioDeUnProductoConIVAPSE.psc AreaRectanguloPSE.psc MetrosACentimetrosPSE.psc X
1 Algoritmo Metros_A_Centimetros
2     definir metros, centimetros Como Real
3
4     escribir "Ingresa cuantos metros convertir"
5     leer metros
6
7     centimetros = metros * 100
8
9     Escribir "Los metros son: " metros "m"
10    escribir " en centimetros son: " centimetros "cm"
11
12
13 FinAlgoritmo
```

```
PSeInt - Ejecutando proceso METROS_A_CENT...
*** Ejecución Iniciada. ***
Ingresa cuantos metros convertir
> 5
Los metros son: 5m
en centimetros son: 500cm
*** Ejecución Finalizada. ***
☐ No cerrar esta ventana ☐ Siempre visible Reiniciar
```

Ilustración 7. Algoritmo METROS_A_CENTIMETROS en PSeInt.

```
1 > #include <stdio.h>
2 #include <stdlib.h>
3
4 > int main() {
5
6     float metros, centimetros;
7
8     printf(format: "Porfavor ingresa los metros a calcular: ");
9     scanf(format: "%f", &metros);
10
11     centimetros = metros * 100;
12
13     printf(format: "Los metros son : %f ,", metros);
14
15     printf(format: "En centimetros son : %f ", centimetros);
16
17 }
18
19
```

```
"D:\mench\Descargas\Ejercicios\EN C\MetrosACentimetrosC.exe"
Porfavor ingresa los metros a calcular:5

Los metros son : 5.000000 ,En centimetros son : 500.000000
Process finished with exit code 0
```

Ilustración 8. Algoritmo METROS_A_CENTIMETROS en C.

PSeint:

Algoritmo PRECIO_IVA

definir precio, total Como Real

escribir "ingrese el precio base del producto:"

leer precio

total = precio * 1.16

Escribir "el precio total con iva incluido es de:" total; escribir "pesos"

C:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    float precioProducto, precioTotal;
```

```
    precioTotal <-0;
```

```
    char Producto[50];
```

```
    printf("Que producto va comprar?");
```

```
    scanf("%s",Producto);
```

```
    printf("Cuanto vale el producto que vas a comprar? ");
```

```
    scanf("%f", &precioProducto);
```

```
    precioTotal = precioProducto * 1.16;
```

```
    printf("el producto es: %s", Producto);
```

```
    printf(" El precio total del producto es : %f", precioTotal);
```

```
    return 0;
```

```
}
```

FinAlgoritmo

Se definió el valor base del producto y el IVA como valores reales, se tomó al 16% del iva como 1.16 para este algoritmo, después se solicitó el precio base del producto deseado y se aplicó la formula “precio * 1.16” y se escribió el resultado, finalmente se cerró el algoritmo

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     float precioProducto, precioTotal;
6
7     precioTotal <-0;
8     char Producto[50];
9
10    printf(format: "Que producto va comprar?");
11    scanf(format: "%s", Producto);
12
13    printf(format: "Cuantos vale el producto que vas a comprar? ");
14    scanf(format: "%f", &precioProducto);
15
16    precioTotal = precioProducto * 1.16;
17
18    printf(format: "el producto es: %s", Producto);
19    printf(format: " El precio total del producto es : %f", precioTotal);
20 }
```

```
"D:\mench\Descargas\Ejercicios\EN C\PrecioDeUnProductoConIvaC.exe"
Que producto va comprar?pizza

Cuantos vale el producto que vas a comprar?100

el producto es: pizza El precio total del producto es : 116.000000
Process finished with exit code 0
|
```

Ilustración 9. Algoritmo PRECIO_IVA en C.

PSeint:

Algoritmo PROMEDIO_3_CALIFICACIONES

definir calificacionI, calificacionII, calificacionIII, promedio Como Real

Escribir "ingrese su primer calificacion:"

leer calificacionI

escribir "ingrese su segunda calificacion:"

leer calificacionII

escribir "ingrese su tercer calificacion:"

leer calificacionIII

$\text{promedio} = (\text{calificacionI} + \text{calificacionII} + \text{calificacionIII}) / 3$

Escribir "su promedio final fue:" promedio

C:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    float calificaciones, Promedio;
```

```
    float calificacion1, calificacion2, calificacion3;
```

```
    printf("Ingresa tu calificacion en el 1er Parcial : ");
```

```
    scanf("%f", &calificacion1);
```

```
    printf("En el segundo : ");
```

```
    scanf("%f", &calificacion2);
```

```
    printf("La tercera : ");
```

```
    scanf("%f", &calificacion3);
```

```
    Promedio = (calificacion1 + calificacion2 + calificacion3) / 3 ;
```

```
    printf("Tu promedio en el semestre es : %.2f", Promedio);
```

```
}
```

Se definieron las variables calificación 1, calificación 2, calificación 3 y promedio como valores reales, de manera consecuente se solicitaron las 3 calificaciones para poder aplicar la formula $(\text{calificación1} + \text{calificación2} + \text{calificación3}) / 3$ para conseguir el promedio, finalmente se escribió el promedio resultante y se cerró el algoritmo

```

1  Algoritmo Promedio_Calificaciones
2  definir Promedio, calificaciones Como real
3  definir calificacion1, calificacion2, calificacion3 Como Real
4
5  Escribir "Ingresa tu calificacion en el 1er parcial : "
6  leer calificacion1
7
8  Escribir "Ingresa en el segundo : "
9  leer calificacion2
10 Escribir "Ingresa en el tercero : "
11 leer calificacion3
12
13 Promedio = (calificacion1 + calificacion2 + calificacion3) / 3
14
15 escribir "Tu promedio del semestre es: " Promedio
16
17
18
19 FinAlgoritmo
20

```

PSInt - Ejecutando proceso PROMEDIO_CALIFICACIONES

```

*** Ejecución Iniciada. ***
Ingresa tu calificacion en el 1er parcial :
> 10
Ingresa en el segundo :
> 8
Ingresa en el tercero :
> 9
Tu promedio del semestre es: 9
*** Ejecución Finalizada. ***

```

Ilustración 10. Algoritmo PROMEDIO_3_CALIFICACIONES en PSInt.

EJERCICIOS DEL 7 DE SEPTIEMBRE DE 2026

Algoritmo ENGANCHE_Y_MENSUALIDADES

Definir Enganche, precio, mensualidades Como Real

precio $\leftarrow$ 0

escribir "ingrese el precio base del vehiculo: "

leer precio

enganche = precio * 0.35

mensualidades $\leftarrow$ (precio - enganche) / 18

Escribir "el enganche total es de: " enganche " pesos"

Escribir "y cada mensualidad seria de: " mensualidades " pesos al mes"

FinAlgoritmo

```
1  Algoritmo ENGANCHE_Y_MENSUALIDADES
2  Definir Enganche, precio, mensualidades Como Real
3  precio  $\leftarrow$  0
4  escribir "ingrese el precio base del vehiculo: "
5  leer precio
6  enganche = precio * 0.35
7  mensualidades  $\leftarrow$  (precio - enganche) / 18
8  Escribir "el enganche total es de: " enganche " pesos"
9  Escribir "y cada mensualidad seria de: " mensualidades " pesos al mes"
10
11 FinAlgoritmo
12
```

```
PSeInt - Ejecutando proceso ENGANCHE_Y_MENSUALIDADES
*** Ejecución Iniciada. ***
ingrese el precio base del vehiculo:
> 100000
el enganche total es de: 35000 pesos
y cada mensualidad seria de: 3611.1111111111 pesos al mes
*** Ejecución Finalizada. ***
```

Ilustración 11. Algoritmo ENGANCHE_Y_MENSUALIDADES en PSeInt.

```
#include <stdio.h>
```

```
int main() {
```

```
    float enganche, precio, mensualidad;
```

```
    printf("Ingresa el precio total del vehiculo: ");
```

```
    scanf("%f", &precio);
```

```
    enganche = precio * 0.35;
```

```
    mensualidad = (precio - enganche) / 18;
```

```
    printf("el total del enganche es: %.2f\n", enganche);
```

```
    printf("el costo de cada mesualidad es: %.2f\n", mensualidad);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>

int main() {
    float enganche, precio, mensualidad;

    printf("Ingresa el precio total del vehiculo: ");
    scanf("%f", &precio);

    enganche = precio * 0.35;
    mensualidad = (precio - enganche) / 18;

    printf("el total del enganche es: %.2f\n", enganche);
    printf("el costo de cada mesualidad es: %.2f\n", mensualidad);

    return 0;
}
```

```
Ingresa el precio total del vehiculo: 100000
el total del enganche es: 35000.00
el costo de cada mesualidad es: 3611.11
```

```
-----
Process exited after 3.458 seconds with return value 0
Presione una tecla para continuar . . . |
```

Ilustración 12. Algoritmo ENGANCHE_Y_MENSUALIDADES en C.

Algoritmo SONIDOS_TEMPERATURA

definir sonidos, temperatura, celcius Como Real
sonidos <- 0
temperatura <- 0
escribir "ingrese la cantidad de sonidos que ha emitido el grillo en un minuto: "
leer sonidos
temperatura <- 50 + ((sonidos - 40) / 4)
celcius <- ((temperatura - 32) * 5) / 9
escribir "la temperatura debe ser de:" temperatura " grados fahrenheit"
ESCRIBIR " y de:" celcius " grados celcius"

FinAlgoritmo

```

1  Algoritmo SONIDOS_TEMPERATURA
2  definir sonidos, temperatura, celcius Como Real
3  sonidos ← 0
4  temperatura ← 0
5  escribir "ingrese la cantidad de sonidos que ha emitido el grillo en un minuto: "
6  leer sonidos
7  temperatura ← 50 + ((sonidos - 40) / 4)
8  celcius ← ((temperatura - 32) * 5) / 9
9  escribir "la temperatura debe ser de:" temperatura " grados fahrenheit"
10 ESCRIBIR " y de:" celcius " grados celcius"
11
12
13 FinAlgoritmo
14

```

*** Ejecución Iniciada. ***
ingrese la cantidad de sonidos que ha emitido el grillo en un minuto:
> 150
la temperatura debe ser de:77.5 grados fahrenheit
y de:25.277777778 grados celcius
*** Ejecución Finalizada. ***

Ilustración 13. Algoritmo SONIDOS_TEMPERATURA en PSeInt.

```
#include <stdio.h>
```

```

int main () {
    float sonidos, temperatura;

    printf(" ingresa el numero de sonidos por minuto que emite el grillo:");
    scanf("%f", &sonidos);
    temperatura = (sonidos/ 4) + 40;

    printf(" la temperatura es: %.2f grados\n", temperatura);

    return 0;
}

```

```

#include <stdio.h>

int main () {
    float sonidos, temperatura;

    printf(" ingresa el numero de sonidos por minuto que emite el grillo:");
    scanf("%f", &sonidos);
    temperatura = (sonidos/ 4) + 40;

    printf(" la temperatura es: %.2f grados\n", temperatura);

    return 0;
}

```

```

ingresa el numero de sonidos por minuto que emite el grillo:150
la temperatura es: 77.50 grados

-----
Process exited after 3.308 seconds with return value 0
Presione una tecla para continuar . . . |

```

Ilustración 14. Algoritmo SONIDOS_TEMPERATURA en C

EXPLICACIÓN

EN PSEINT:

ALGORITMO 1:

Primero se declararon las respectivas variables que fueron la cantidad de sonidos y la temperatura como reales, posteriormente se definió el valor de la temperatura y de la cantidad de sonidos a mayor que 0, luego se solicita la cantidad de veces que el grillo emitió sonidos en un minuto, se aplicó la fórmula $\text{temperatura} \leftarrow 50 + ((\text{sonidos} - 40) / 4)$ para sacar la temperatura en Fahrenheit y finalmente se cerró el algoritmo

Algoritmo 2:

Primero se declararon las variables de precio, enganche y mensualidad como reales, posteriormente se solicitó el precio total del vehículo, después se calculó el 35% de enganche mediante la fórmula $\text{enganche} \leftarrow \text{precio} * 0.35$, posteriormente se calculó el importe de las 18 mensualidades sin intereses con la fórmula $\text{mensualidad} \leftarrow (\text{precio} - \text{enganche}) / 18$, y finalmente se mostraron el enganche y la mensualidad.

EN C:

Algoritmo 1:

Primero se declararon las variables de sonidos y temperatura como float, posteriormente se solicitó la cantidad de sonidos que emitió el grillo en un minuto, después se aplicó la fórmula $\text{temperatura} \leftarrow 50 + ((\text{sonidos} - 40) / 4)$ para obtener la temperatura en Fahrenheit y finalmente se mostró el resultado y se cerró el algoritmo.

Algoritmo 2:

Primero se declararon las variables de precio, enganche y mensualidad como float, posteriormente se solicitó el precio total del vehículo, después se calculó el 35% de enganche mediante la fórmula $\text{enganche} \leftarrow \text{precio} * 0.35$, posteriormente se calculó el importe de las 18 mensualidades sin intereses con la fórmula $\text{mensualidad} \leftarrow (\text{precio} - \text{enganche}) / 18$, y finalmente se mostraron el enganche y la mensualidad.

CONCLUSIÓN

En conclusión, la Unidad I: Introducción a la Algoritmia permite comprender los conceptos fundamentales necesarios para resolver problemas de manera lógica, ordenada y eficiente. A través del estudio de la computabilidad y del concepto de algoritmo, se reconoce la importancia de establecer una serie de pasos finitos y precisos que permitan alcanzar una solución.

Asimismo, conocer los elementos que conforman un algoritmo y los diferentes tipos de datos proporciona las bases para organizar y manipular correctamente la información. La representación mediante diagramas de flujo facilita la comprensión de los procesos, ya que permite visualizar de manera gráfica las instrucciones y decisiones que forman parte de una solución.

El aprendizaje de las estructuras básicas —secuencial, condicional e iterativa— permite construir algoritmos capaces de resolver diferentes situaciones, desde problemas sencillos hasta procesos que requieren tomar decisiones o repetir determinadas acciones. Finalmente, la resolución de problemas básicos ayuda a desarrollar el pensamiento lógico y la capacidad de analizar una situación para encontrar una solución adecuada.

Por lo tanto, la introducción a la algoritmia constituye una base esencial para el aprendizaje de la programación, ya que antes de crear un programa es necesario aprender a analizar problemas, organizar ideas y diseñar soluciones mediante algoritmos claros y eficientes.

REFERENCIAS

En este apartado se agregarán todas referencias en formato APA, según las investigaciones realizadas.

Maluenda, R. (2024, diciembre 30). Qué es un algoritmo informático: características, tipos y ejemplos. Profile Software Services.

<https://profile.es/blog/que-es-un-algoritmo-informatico/>

© 2013-2026 Enciclopedia Concepto. Todos los derechos reservados.

Algoritmo en informática. (2018, abril 9). Concepto. <https://concepto.de/algoritmo-en-informatica/>

Representación de Algoritmos: Pseudocódigo y Diagramas de Flujo. (s/f).

Algoreducation.com. Recuperado el 24 de agosto de 2026, de

<https://cards.algoreducation.com/es/content/oErAM-F1/algoritmos-pseudocodigo-diagramas-flujo>

Fernando Palmero, I.S.C, M.A, & Perfil, V. T. mi. (n.d.). Aprendizaje de Algoritmos - TICCAD. Blogspot.com. Retrieved September 10, 2026, from

<https://aprendizajealgoritmos.blogspot.com/2011/02/estructuras.html>

Teorema del programa estructurado - Creador de Páginas ilustradas. (s/f).

Proyectodescartes.org. Recuperado el 26 de agosto de 2026, de

<https://proyectodescartes.org/escenas-aux/CursoObjetosDescartesIA/sesion5/>

(S/f). Onmex.mx. Recuperado el 26 de agosto de 2026, de

<https://onmex.mx/tecnologia-y-desarrollo/estructuras-condicionales-que-son-y-para-que-sirven/>

Teorema del programa estructurado - Creador de Páginas ilustradas. (s/f).

Proyectodescartes.org. Recuperado el 26 de agosto de 2026, de

<https://proyectodescartes.org/escenas-aux/CursoObjetosDescartesIA/sesion5/>

(S/f). Onmex.mx. Recuperado el 26 de agosto de 2026, de

<https://onmex.mx/tecnologia-y-desarrollo/estructuras-condicionales-que-son-y-para-que-sirven/>